

Differences Between Functions and Function Templates

Abstract

There are a number of differences between functions and function templates in C++. This was highlighted during the discussion around inclusion of abbreviated function template syntax from the Concepts TS in the C++ WP. This informal list of the main ways in which functions and function templates differ was assembled for that discussion, so it is now published as a paper for use in a more general context.

Contents

1. [List of Differences](#)
 - [Forwarding/rvalue references](#)
 - [Definition visible at point of use](#)
 - [The One Definition Rule](#)
 - [Static Locals](#)
 - [Function addresses](#)
 - [Local types are distinct](#)
 - [Template argument deduction](#)
 - [Overload resolution](#)
 - [Selection via `f<>\(/\*...\*/\)`](#)
 - [Specialization](#)
 - [Special members](#)
 - [Deferred checking](#)
 - [Delayed checking of default arguments](#)
 - [Dependent names](#)
 - [Specialize or Overload](#)
 2. [Revision History](#)
 3. [Acknowledgements](#)
-

List of Differences

Here is a list of differences: there is no particular significance to their order.

Forwarding/rvalue references

A declaration of a function argument with type `T&&` is an rvalue reference.

A declaration of a function template argument with type `T&&` for some template argument `T` is a *forwarding* reference.

[Example:

```
struct T{};

void foo(T&& t); // rvalue reference

template <typename T>
void bar(T&& t); // forwarding reference

int main()
{
```

```

    T t{};
    foo(t); // error: cannot bind rvalue ref to t
    bar(t); // OK: reference collapse to T&
}

```

-- end example]

Definition visible at point of use

A function and function template can be declared but not defined in a header. At link time a definition of the entity must be provided.

For a function this simply involves providing a definition in one of the TUs.

This is in general infeasible for a function template as a specialization or explicit instantiation must be provided for **every** instantiation that might be invoked.

(Additionally, such explicit specializations must be *declared* in every translation unit, although absence may not be diagnosed by all implementations.)

This can be a problem as code changes: you might for example start with a function in an anonymous namespace, used in a single .cpp file.

Then you want to use it elsewhere, so you move the definition into a public namespace, and put the declaration in a header so you can use it in a second .cpp file.

The One Definition Rule

A function definition can only be provided **once** in a given program (unless the function is marked `inline`).

A function template can be defined in *multiple* translation units, and the linker will pick one copy of each actual instantiation.

Static Locals

In a function there is a single instance of a function-scope static; for a function template there is one instance per instantiation.

[Example:

```

inline void foo()
{
    static object theObject; // only one instance
}

template <typename T>
void foo()
{
    static object theObjects; // one instance per T
}

```

-- end example]

Function addresses

There a single address for a function, for a function template there is potentially a separate address for each instantiation.

A (non-inlined) function has a single instantiation in a program; multiple instantiations of a (non-inlined) function template increase the code size.

Local types are distinct

If a local type is defined within a function template each instantiation of the function template produces a *distinct* type.

[*Example:*

```
template<class T>
auto f(const T&) {
    struct A {};
    return A{};
}

void g(short s) {
    auto a=f(s);
    a=f(s+1);    // error -- type mismatch
}
```

-- end example]

Template argument deduction

vs. implicit conversion sequences

Deduction of function template arguments does not consider implicit conversion sequences.

[*Example:*

```
struct X{};
struct Y : X {};

void foo(X, X);

template<typename T>
void bar(T, T);

int main()
{
    X x;
    Y y;
    foo(x, y); // OK
    bar(x, y); // error
}
```

-- end example]

or a different case:

[*Example:*

```
struct X{X(int) {} };

bool operator==(X, X);

bool zero(X x) {return x == 0;} // OK

template<typename T>
struct Y { Y(T) {} };

template<class T>
bool operator==(Y<T>, Y<T>); // non-member allows conversions, right?

bool zero(Y<int> y) {return y == 0;} // error: deduction failed
```

-- end example]

Initializer lists

Templates do not allow deduction of `initializer_list`.

[*Example:*

```
#include <initializer_list>

void f(std::initializer_list<int> i) {}

template <typename T>
void ft(T t) {}

int main()
{
    f({1, 3, 5});

    ft({1, 3, 5}); // error: can't deduce initializer_list
}
```

-- end example]

Contextual cases

Conversion functions

Deduction is required to use a conversion function template. [*Example:*

```
auto f() {
    auto x = [](int){};
    auto y = [](auto){};

    auto z = +y; // error: no conversion found
    return +x;  // OK
}
```

-- end example]

Pointers to overload sets

Taking the address of a function returns a unique value.

Taking the address of a function template does not return a single value.

This makes using function pointers with function templates trickier than with functions as the instantiation required must be explicitly specified.

This can be done by giving the desired template arguments for the specialization desired.

When the template arguments are *deducible* an alternative is using a pointer to a function with the desired argument types to select the correct instantiation.

[*Example:*

```
struct X {};

void foo(X, X);
X make_foo();

template<typename T>
void bar(T, T);

template<typename T>
T make_bar(T, T);

int main()
{
    auto foo_ptr = &foo; // OK
    auto make_foo_ptr = &make_foo; // OK
```

```

    auto bar_ptr = &bar; // error
    auto make_bar_ptr = &make_bar; // error

    auto bar_ptr1 = &bar<X>; // OK
    auto make_bar_ptr1 = &make_bar<X>; // OK

    void (*bar_ptr2)(X, X) = &bar; // OK, template arguments deduced
}

```

-- end example]

Function templates as arguments

There are differences between using functions and using function templates as arguments to function templates.

[Example:

```

#include <iostream>

template<class T>
T f_template(const T &arg){
    return arg;
}
int f_int(const int &arg){
    return arg;
}
auto f_lambda = [](const int &arg){
    return arg;
};

auto f_glambda = [](const auto &arg){
    return arg;
}

template<class F, class T>
auto call_me(F f, const T&arg){
    return f(arg);
}

int main(int, char**){
    std::cout << call_me(f_int, 42) << "\n";           // works
    std::cout << call_me(f_lambda, 42) << "\n";        // works
    std::cout << call_me(f_glambda, 42) << "\n";        // generic lambda works
    std::cout << call_me(f_template, 42) << "\n";       // template doesn't work
}

```

-- end example]

Operator delete template

An operator delete template behaves differently from a non-template.

[Example:

```

struct A {
    A() {throw 0;}
    void* operator new(std::size_t,int,short);
    template<class T>
    void operator delete(void*,T,T);
};

void f() {new (0,0) A;} // no deallocation function called

```

-- end example]

Overload resolution

SFINAE

Function templates whose declarations cannot be substituted into are simply removed from the overload set. Functions are either present or not present and there are no techniques to remove them from consideration for particular types of argument.

Best viable function

The rules for determining the best viable function for overloaded functions and function templates are different.

[*Example:*

```
template<class T>
void f(T*);           // #1
template<class T>
void f(T**);          // #2

void g(int*);          // g ~ f<int>
void g(int**);         // and yet...

void h() {
    f<int>(nullptr);    // OK: calls #2
    g(nullptr);        // error: ambiguous
}
```

-- *end example*]

Tie-breaking

In the case of a tie-break between a function and a function template the function wins.

Explicit specializations

The interaction between overloading (based on the primary template) and explicit specializations can be confusing.

[*Example:*

```
template<class T>
void f(const T&);      // #1
template<> void f(const int&); // #2: specializes #1

void f(volatile int&); // #3

void g(int i) {f(i);}  // calls #3
```

-- *end example*]

If we added one (more) template parameter to each of the three declarations of f, the call would become ambiguous.

Selection via `f<>(...)`

For a function template you can select the template explicitly by providing an empty template argument list.

[*Example:*

```
void foo(int);

template<typename T>
void foo(T);

int main()
{
    foo(1); // Selects function (best match)
```

```
    foo<>(1); // Selects function template
}
```

-- end example]

Specialization

For a function template you can provide explicit specializations (you obviously cannot for a non-template.)

[Example:

```
template<typename T>
void bar(T);
```

```
template<>
void bar(double) {}
```

```
int main()
{
    bar(1.0);
}
```

-- end example]

For a function template you can specify an instantiation explicitly by providing the template arguments.

[Example:

```
template<typename T>
void bar(T);
```

```
int main()
{
    bar<int>(1.0);
}
```

-- end example]

Special members

A function template cannot be a special member function, even if some specialization of it meets the requirements.

[Example: (from [class.copy.ctor] p5)

```
struct S {
    template<typename T> S(T);
    S();
};
```

```
S g;
```

```
void h() {
    S a(g); // does not instantiate the member template to produce S::S<S>(S);
           // uses the implicitly declared copy constructor
}
```

-- end example]

Deferred checking

A function is compiled when the definition is reached.

A function template has two phases of compilation, once when the definition is reached and once when the function is instantiated.

This can mean there is no guarantee of validity checking until instantiation.

[Example:

```
template<class T>
bool equal(const Container<T> &c, const T *a) {
    for(int i=0; i<c.size(); ++i)
        if(c[i]!=a) return false;
    return true;
}
```

-- end example]

The expression `c[i]!=a` is comparing a `T` against a `T*` and will (almost certainly) fail at instantiation. (And even if it instantiates, it's unlikely to be the correct semantics!)

Deferred checking - `static_assert`

In a function a `static_assert` triggers at definition time.

In a function template a (dependent) `static_assert` triggers at instantiation time.

[Example:

```
void foo(int i)
{
    static_assert(sizeof(i) == 88); // asserts: (typo for '8')
}

template<typename T>
void bar(T t)
{
    static_assert(sizeof(int) == 88); // ill-formed, no diagnostic required (non-dependent)
    static_assert(sizeof(T) == 88); // asserts when instantiated (dependent)
}
```

-- end example]

Delayed checking of default arguments

The handling of default arguments is different as they are checked at declaration time for functions, but at instantiation for function templates.

[Example:

```
struct X {};

void foo(X arg = 42); // error: even if default argument never used

template<typename T>
void bar(T arg = 42); // OK

int main()
{
    X x;
    bar(x); // OK
    bar<X>(); // error: only if default argument used
}
```

-- end example]

Dependent names

Access to declarations after function (template) declaration

[Example:


```
// Taken from [temp.res] p10, slightly simplified

void f(char);
enum E { e };

template<class T> void g(T t) {
    f(1); // f(char)
    f(t); // dependent
}

void f(E);

void h() {
    g(e); // will cause one call of f(char) followed by one call of f(E)
}

-- end example]
```

The equivalent case for a function has *different* behaviour:

[Example:

```
void f(char);
enum E { e };

void g(E t) {
    f(1); // f(char)
    f(t); // f(char) !
}

void f(E);

void h() {
    g(e); // will cause two calls of f(char)
}

-- end example]
```

Necessity of typename and template

When accessing dependent names additional syntax is required in certain cases.

Note that while [P0634R3](#) (applied to the working paper in JAX '18) *reduces* the number of places where typename is required, it does not eliminate them all.

For example, that paper gives an example where omitting typename changes the meaning of the code:

```
template<typename T> void f() {
    void (*pf)(T::X); // Variable pf of type void* initialized with T::X
};
```

If typename is added:

```
template<typename T> void f() {
    void (*pf)(typename T::X); // Variable pf of type pointer to function taking T::X and returning void.
};
```

There are also cases where template must be used to avoid parsing ambiguity with the less than operator.

[Example:

```
// Taken from [temp.names] p3, slightly simplified

struct X {
    template<std::size_t> X* alloc();
    template<std::size_t> static X* adjust();
};

template<class T> void f(T* p) {
```

```

T* p1 = p->alloc<200>(); // ill-formed: < means less than
T* p2 = p->template alloc<200>(); // OK: < starts template argument list
}

```

-- end example]

Specialize or Overload

Whether to specialize or overload depends on whether the first declaration is a function or a function template.

With the "Constrained auto" syntax adopted in [P1141](#):

[Example:

```

// library code
template<class T>
concept Container=/*...*/;

struct Widget {};
struct Button : Widget {};

// user code
void f(Container auto&) {}
template<> void f(std::vector<int>&) {} // OK

void g(Widget&) {}
template<> void g(Button&) {} // error

```

-- end example]

With the abbreviated syntax from the Concepts TS the difference between f and g becomes less visible:

[Example:

```

// user code
void f(Container&) {}
template<> void f(std::vector<int>&) {} // OK

void g(Widget&) {}
template<> void g(Button&) {} // error

```

-- end example]

Revision history

- P1392R0 (this document): "Specialize or Overload" revised following adoption of P1141 into the C++ Working Paper.
Removal of "multiple overloads with satisfied constraints" as this is specific to the Concepts TS merging discussion.
Other additional minor edits.
 - Pre-publication: A first, informal, version of this paper was produced in Jun 2018 for use during discussion in Rapperswil and attached to the EWG Wiki.
-

Acknowledgements

Thanks to those who gave additional suggestions and examples, including:

- Richard Smith
- Hubert Matthews
- Sam Finch
- Thomas Russell

Any *errors* in this paper are the responsibility of the paper authors.